ASSIGNMENT 1

Introduction to Web Development

Submitted by: **Sohum Sabhani**

Course: **Introduction to Web Development**

Topic: **Client-Server Model, Web Architecture & Development Tools**

Contents

- Q1 — Frontend vs Backend vs Full-Stack Development
- Q2 — Client-Server Model Diagram
- Q3 — How a Browser Requests & Displays a Web Page
- Q4 — Tools for a Web Development Environment
- Q5 — Web Servers and Examples
- Q6 — Roles: Frontend / Backend Developer / DBA
- Q7 — Installing & Configuring VS Code
- Q8 — Static vs Dynamic Websites
- Q9 — Web Browsers & Rendering Engines
- Q10 — Labeled Web Architecture Flow Diagram

Q1. Difference between Frontend, Backend, and Full-Stack Development

Web development is broadly divided into three areas based on which part of an application a developer works on. Each area uses a different set of technologies and focuses on a different layer of the system.

Aspect	Frontend Development	Backend Development	Full-Stack Development
Definition	Builds the part of the site users see and interact with.	Builds the server-side logic, database and APIs behind the scenes.	Works across both frontend and backend.
Technologies	HTML, CSS, JavaScript, React, Vue	Node.js, Python, Java, PHP, SQL/NoSQL	Combination of both sets of tools
Focus	UI/UX, layout, responsiveness	Business logic, security, data storage	End-to-end application delivery
Real-world example	Designing the product page layout on Amazon	Processing a payment and updating inventory in Amazon's servers	A developer who builds an entire e-commerce site solo

In short, a **frontend developer** is like an architect designing the visible rooms of a house, a **backend developer** is like the plumber and electrician working behind the walls, and a **full-stack developer** can do both jobs.

Q2. Diagram: How the Client-Server Model Works in Web Architecture

The client-server model is the foundation of web architecture. The **client** (browser) sends a request over the **internet** to a **server**, which processes it and sends back a response.

Fig 1: Client-Server Model in Web Architecture

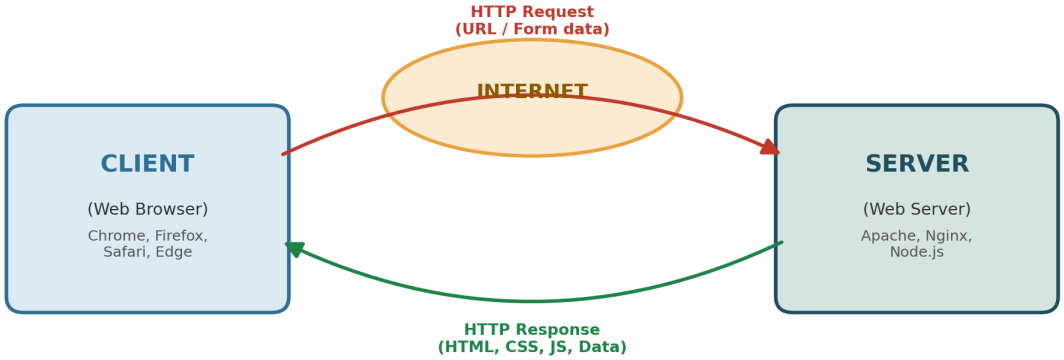


Fig 1: Client sends an HTTP request; server processes and returns an HTTP response.

- **Client:** Initiates the request (e.g., typing a URL or clicking a link).
- **Internet:** Carries the request/response using protocols like HTTP/HTTPS.
- **Server:** Listens for requests, processes them, and returns the requested resource.

Q3. How a Browser Requests and Displays a Web Page

When a user enters a URL, the browser goes through a defined sequence of steps to fetch and render the page:

- **URL Parsing:** The browser breaks the URL into protocol, domain and path.
- **DNS Lookup:** The domain name is resolved to an IP address using DNS.
- **TCP/TLS Connection:** A connection is established with the server (secured via TLS for HTTPS).
- **HTTP Request:** The browser sends a GET request for the page.
- **Server Response:** The server returns HTML (and later CSS, JS, images) with a status code (e.g., 200 OK).
- **Parsing & DOM Construction:** The browser parses HTML into the DOM tree and CSS into the CSSOM.
- **Rendering:** The rendering engine combines DOM + CSSOM into a render tree, calculates layout, and paints pixels on screen.
- **JavaScript Execution:** Scripts run and can further modify the DOM, triggering re-renders.

Q4. Tools Required to Set Up a Web Development Environment

Tool	Purpose
Code Editor / IDE (VS Code, Sublime Text)	Writing and editing HTML, CSS, and JavaScript code with syntax highlighting and extensions.
Web Browser (Chrome, Firefox)	Running and testing the website; DevTools help debug HTML/CSS/JS.
Version Control (Git & GitHub)	Tracking changes to code and collaborating with a team.
Web Server (Node.js, XAMPP, Live Server)	Serving pages locally to simulate a real hosting environment.
Package Manager (npm / yarn)	Installing and managing external libraries and frameworks.
Browser DevTools	Inspecting elements, debugging scripts, checking network requests.
Terminal / Command Line	Running build tools, servers, and Git commands.
Design Tools (Figma)	Creating UI mockups before development begins.

Q5. What is a Web Server? Examples of Commonly Used Servers

A **web server** is software (often running on dedicated hardware) that stores, processes, and delivers web pages to clients upon request, typically using the HTTP/HTTPS protocol. It listens for incoming requests, retrieves the appropriate resource (static file or dynamically generated content), and sends it back to the requesting browser.

Commonly Used Web Servers

- **Apache HTTP Server:** One of the oldest and most widely used open-source web servers.
- **Nginx:** Known for high performance and efficient handling of concurrent connections; also used as a reverse proxy.
- **Microsoft IIS (Internet Information Services):** Windows-based server commonly used with .NET applications.
- **Node.js (Express):** A JavaScript runtime used to build lightweight, custom web servers.
- **LiteSpeed:** A high-performance commercial alternative to Apache.

Q6. Roles of Frontend Developer, Backend Developer, and Database Administrator

Role	Key Responsibilities
Frontend Developer	Builds the user interface using HTML, CSS, and JavaScript; ensures responsiveness, accessibility, and a smooth user experience; integrates with backend APIs.
Backend Developer	Designs server-side logic, builds and maintains APIs, handles authentication, business rules, and integrates with the database.
Database Administrator (DBA)	Designs the database schema, manages data storage, ensures data integrity, performs backups, optimizes queries, and manages security/access control.

Q7. Installing and Configuring VS Code for HTML, CSS, and JavaScript

Steps to set up Visual Studio Code as a web development environment:

- Download VS Code from the official site **code.visualstudio.com** for your operating system.
- Run the installer and follow the setup wizard (Windows: check 'Add to PATH' and 'Open with Code' options).
- Launch VS Code and open the Extensions panel (Ctrl+Shift+X).
- Install useful extensions: **Live Server** (auto-refreshing local preview), **Prettier** (code formatting), **ESLint** (JavaScript linting), and **HTML CSS Support**.
- Create a project folder with an **index.html**, a **style.css**, and a **script.js** file.
- Right-click index.html and select 'Open with Live Server' to preview the page in the browser with live reload.

Note: A screenshot of the completed VS Code setup (editor with the extensions installed and a sample project open) should be inserted here as required by the question. Please attach your personal screenshot in this section before final submission, since it must reflect your own machine's setup.

Q8. Difference between Static and Dynamic Websites

Aspect	Static Website	Dynamic Website
Content	Fixed; same for every visitor	Changes based on user, time, or database data
Technology	HTML, CSS, minimal JS	HTML/CSS/JS + server-side language (PHP, Node.js, Python) + database
Update process	Requires manually editing files	Content updates automatically via database/CMS
Speed	Very fast (no server processing)	Slightly slower (server processing + DB queries)
Example	A simple portfolio or brochure site	Facebook, Amazon, Gmail — content changes per user

Q9. Five Web Browsers and Their Rendering Engines

Different browsers use different rendering engines to convert HTML/CSS into the visuals users see, which can cause minor differences in how pages display.

Browser	Rendering Engine	Notes
Google Chrome	Blink	Fast, widely used; also powers Edge, Opera, Brave

Browser	Rendering Engine	Notes
Mozilla Firefox	Gecko	Open-source engine developed by Mozilla; strong standards support
Apple Safari	WebKit	Default browser on macOS/iOS; Blink was originally forked from WebKit
Microsoft Edge	Blink (Chromium-based)	Switched from Microsoft's own EdgeHTML to Blink in 2020
Opera	Blink (Chromium-based)	Built on the same open-source Chromium project as Chrome

Rendering engines mainly differ in how strictly they follow web standards, their JavaScript engine performance (e.g., Chrome's V8 vs Firefox's SpiderMonkey), and support for newer CSS/HTML features — which is why developers test pages across multiple browsers.

Q10. Labeled Diagram: Basic Web Architecture Flow

A typical web application flow moves through four layers: the client sends a request to the server, the server consults the API layer for business logic, which in turn queries the database and returns data back up the chain to the client.

Fig 2: Basic Web Architecture Flow — Client, Server, API, Database

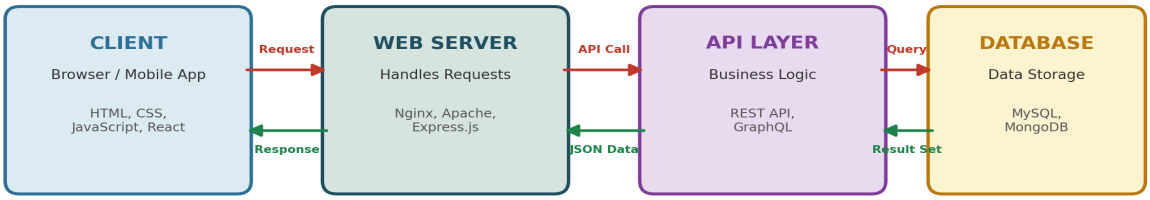


Fig 2: Client → Server → API → Database, with data flowing back in the response.

- **Client:** Sends the initial request from the browser or app.
- **Server:** Receives the request and routes it appropriately.
- **API Layer:** Executes business logic and communicates with the database.
- **Database:** Stores and returns the requested data (via SQL or NoSQL queries).

— End of Assignment 1 —